

P2119R0

Feedback on P1708: Simple Statistical Functions

Published Proposal, 2020-02-13

Author:

Jolanta Opara (Mobica Limited sp.z o.o. Oddział w Polsce)

Audience:

SG19

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The paper contains comments about [\[P1708R0\]](#), [\[P1708R1\]](#) and [\[P1708R2\]](#). It also provides a comparison of the analogical functionalities provided in python, R, SAS and Matlab and some comments from a users' perspective.

Table of Contents

1	Motivation
2	Users' perspective
3	Common expectations for all statistics
4	Free-standing statistical functions
4.1	Mean
4.1.1	Definition
4.1.2	Users' perspective
4.1.3	Interfaces in other languages
4.2	Median
4.2.1	Definition
4.2.2	Users' perspective
4.2.3	Interfaces in other languages
4.3	Quantiles
4.3.1	Definition
4.3.2	Users' perspective
4.3.3	Interfaces in other languages
4.4	Mode
4.4.1	Definition
4.4.2	Users' perspective
4.4.3	Interfaces in other languages
4.5	Standard deviation
4.5.1	Definition
4.5.2	Users' perspective
4.5.3	Interfaces in other languages
4.6	Variance
4.6.1	Users' perspective
4.6.2	Interfaces in other languages
5	Grouped statistics
5.1	Expectations from the users' perspective
5.2	Users' perspective
6	Some use-cases
7	Summary
8	Document history
9	Acknowledgements
	References
	Normative References
	Informative References

§ 1. Motivation

This paper reviews [\[P1708R0\]](#), [\[P1708R1\]](#) and [\[P1708R2\]](#); Simple Statistical Functions and summarizes the discussions about the functionality. All the reviews are mentioned because the approach towards the functionality changes between the reviews.

The idea of the paper is to provide an overview of users' expectations towards the statistical functions and a comparison of analogical features available in other programming languages. The paper does not propose any syntax. Subsections about users' perspective contain opinions only.

§ 2. Users' perspective

Simple statistical functions are available not only in python, as it was mentioned in [\[P1708R0\]](#), but also in R, S, SAS, Matlab and many other languages used by statisticians.

Potential users expect both free-standing simple statistical functions (for simplicity) and some way to calculate them all in one pass, or as fewer passes as possible, over a container (for speed). Additionally, they expect user-friendly, intuitive and extendable interface for both use-cases. Preferably, it should be easy to use for an inexperienced user and easy to customize for more sophisticated use-cases.

Typically, users need to calculate one statistic with the default parameters. It seems excessive to require an additional object in these cases. The requirement that the collection is sorted is unintuitive. Hence, it is unexpected as the default option and it may be error-prone. Moreover, the necessity to sort the collection before the calculation of statistics may lead to computational overhead.

§ 3. Common expectations for all statistics

- both iterator-based and range-based versions
- free-standing functions that are simple in use
- possibility to compute multiple statistics of the same sample in a computationally effective and intuitive way, consistent with the interface of free-standing functions
- possibility to choose internal representation/return type for the calculation (e.g. the mean of integers can be fractional, so should computations be made on ints, floats or doubles?)
- customization of NaNs and missing data: yes/no, functor, list of values (?). This is a common issue in many statistical domains.
- transformation possibility

The last two points seem to be achievable with ranges, for example:

```
range | std::views::filter(not_nan_in_key) | std::views::transform(value_only)
```

or depending on the user needs

```
range | std::views::transform(key_only) | std::views::filter(not_nan)
```

§ 4. Free-standing statistical functions

Some statistical functions have been proposed in [\[N1668\]](#): A Proposal to add Mathematical Functions for Statistics to the C++ Standard Library, but the proposal did not cover the topics described in the P1708 paper. The N1668 proposes functions to calculate the statistical properties of common probability distributions (e.g. quantiles, densities of the normal distribution with chosen parameters) whereas P1708 proposes functions to calculate statistical properties of a sample.

§ 4.1. Mean

§ 4.1.1. Definition

The arithmetic mean of the sample. The definition is given in [\[P1708R2\]](#)

§ 4.1.2. Users' perspective

The most commonly used mean function is an arithmetic mean defined in P1708. However, it could be extended to compute the trimmed arithmetic mean, which is robust against outliers. It would be useful to have it supported under the same or different function name.

§ 4.1.3. Interfaces in other languages

Language	Python	R	SAS
Syntax	<pre>mean([data-set])</pre>	<pre>mean(x, trim = 0, na.rm = FALSE, ...)</pre>	<pre>MEAN(argument-1 <, ... a</pre>
Parameters	[data-set]: List or tuple of a set of numbers.	x: An R object. trim: the fraction (0 to 0.5) of observations to be trimmed from each end of x before the mean is computed. na.rm: a logical value indicating whether NA values should be stripped before the computation proceeds. ...: further arguments passed to or from other methods.	argument: specifies a numeric constant expression.
Returns	Sample arithmetic mean of the provided data-set.	Sample arithmetic mean of the (optionally: non-missing) provided data-set.	Sample arithmetic mean of the non-missing provided data-set.
Restrictions	Numeric values must be passed as parameter.	Numeric/logical vectors and date, date-time and time interval objects. Complex vectors are allowed for trim = 0, only.	At least one non-missing argument is required.
Error handling	Exceptions TypeError	If x is not logical (coerced to numeric), numeric (including integer) or complex, NA_real_ is returned, with a warning.	The function returns a missing value.
Details	python_doc_mean geeksforgeeks_mean	r_doc_mean	Matlab_doc SAS_doc_mean

All the languages listed above, except python, provide a way to exclude missing value. It is worth to mention that all the languages have a NaN “type” for missing data. The interfaces that do not have a trim parameter provide separate functions for a trimmed mean.

§ 4.2. Median

§ 4.2.1. Definition

Median is a quantile with probability 0.5. The definition of a quantile will be provided in section 4.3. One of the definitions of median is described in [\[P1708R1\]](#).

§ 4.2.2. Users' perspective

It is a quantile so one can either treat it this way, with all the customization points available for quantiles, or reduce the use-cases for std::median to the definition from [\[P1708R1\]](#) and types that support the required mathematical operations.

In other programming languages median returns a single value. For details about python, R, Matlab and SAS, see the comparison in the following subsection. Moreover, in [\[UNUR_2016\]](#) A. Sinan Unur described median calculation results

from perl, Stata, R, Octave, WolframAlfa and Boost library and they all returned a single number, so it would be counter-intuitive to return a tuple in the same case in the C++ standard library as proposed in [\[P1708R2\]](#).

§ **4.2.3. Interfaces in other languages**

Language	Python	R
Syntax	<pre>statistics.median(data) numpy.median(a, axis=None, out=None, overwrite_input=False, keepdims=False)</pre>	<pre>median(x, n</pre>
Parameters	a: array_like object axis: {int, sequence of int, None}, optional Axis or axes along which the medians are computed. out: ndarray, optional Alternative output array in which to place the result. overwrite_input: bool, optional If True, then allow use of memory of input array a for calculations. keepdims: bool, optional If this is set to True, the axes which are reduced are left in the result as dimensions with size one.	x: an object for which the median is defined, or a numerical value na.rm: a logical value indicating whether NA values should be stripped before the computation proceeds ...: potentially further arguments not used in the default
Returns	statistics: statistics.median return the median (middle value) of numeric data, using the common “mean of middle two” method numpy: ndarray A new array holding the result. If the input contains integers or floats smaller than float64, then the output data-type is np.float64. Otherwise, the data-type of the output is the same as that of the input. If out is specified, that array is returned instead..	The default method of the same type as x, or integer of even length will be double. If there are no values or all values are NA, NA is returned. If there are NA values present, the default is to strip them out.
Restrictions	statistics.median: data can be a sequence or iterable. The input array will be modified by the call to median.	See: details
Error handling	If data is empty, StatisticsError is raised	the result is NA of the same type as x
Details	python_doc_median	R_doc_median

All the languages listed above, except python, provide a way to exclude missing values. It is worth to mention that all the languages have a NaN “type” for missing data. Also, all the interfaces return one value as a result.

§ **4.3. Quantiles**

The only quantile mentioned in P1708 is median, but I think that quantile function is worth considering as an input to discussion about a median of non-numeric types.

§ 4.3.1. Definition

Quantiles are cut points dividing the range of a probability distribution into continuous intervals with equal probabilities, or dividing the observations in a sample in the same way. See: [\[HYNDMAN+FAN_1996\]](#)

§ 4.3.2. Users' perspective

In [\[HYNDMAN+FAN_1996\]](#) Rob J. Hyndman and Yanan Fan discussed nine algorithms of quantile calculation. They are summarized in [R_doc_quantile](#) in Type section. Types 1 and 3 can be applied to non-numeric types as they do not require interpolation and so they are adequate for discontinuous sample quantiles.

§ 4.3.3. Interfaces in other languages

Language	Python
Syntax	<pre>numpy.quantile(a, q, axis=None, out=None, overwrite_input=False, interpolation='linear', keepd</pre>
Parameters	a: array_like input array or object that can be converted to an array. q: array_like of float Quantile or sequence of quantiles to compute, which must be between 0 and 1 inclusive. axis{int, tuple of int, None}: Axis or axes along which the quantiles are computed. out: ndarray Alternative output array in which to place the result. overwrite_inputbool: If True, then allow the input array a to be modified by intermediate calculations, to save memory. interpolation{'linear', 'lower', 'higher', 'midpoint', 'nearest'}: This optional parameter specifies the interpolation method to use wh quantile lies between two data points i < j:
Returns	scalar or ndarray If q is a single quantile and axis=None, then the result is a scalar. If multiple quantiles are given, first axis of the result corresponds to
Restrictions	See: details
Error handling	not specified
Details	python_doc_quantile

Every syntax described above provides a way to customize how the quantiles are estimated. It can be configured by choosing an approximation method (Python, Matlab), or algorithm from [\[HYNDMAN+FAN_1996\]](#) (SAS, R).

§ 4.4. Mode

§ 4.4.1. Definition

The definition is provided in [\[P1708R2\]](#)

§ 4.4.2. Users' perspective

Despite a possibly linear complexity the function may have huge space requirements for example when a container is big and contains unique numbers. It would be good to have some policy that allows returning a single value in case of multiple modes.

§ 4.4.3. Interfaces in other languages

Language	Python	R	SAS
Syntax	<pre>statistics.mode(data) scipy.stats.mode(a, axis=0, nan_policy='propagate')</pre>	<pre>Mode(x, na.rm = FALSE)</pre>	not provided
Parameters	data: discrete or nominal data a: array_like n-dimensional array of which to find mode(s). axis: int or None, Axis along which to operate. None means full matrix nan_policy: {'propagate', 'raise', 'omit'} Nan handling	x: an object for which a method has been defined, or a numeric vector containing the values na.rm: a logical value indicating whether NA values should be stripped before the computation proceeds.	—
Returns	statistics: Return the single most common data point from discrete or nominal data. If there are multiple modes with the same frequency, returns the first one encountered in the data scipy: If there is more than one such value, only the smallest is returned. Returns mode: ndarray Array of modal values and count: ndarray Array of counts for each mode.	Returns the most frequent value. If there are more than one, all of them will be returned in a vector.	—
Restrictions	statistics: assumes discrete data and returns a single value	See: details	—
Error handling	statistics: StatisticsError is raised when more than one mode was found.	NaNs can be ignored	—
Details	python_statistics_mode	R_doc_mode	—

Not each of the above interfaces provides a mode function. Python always returns a single value (however, there is a separate function that returns multiple values), Matlab can provide both single (lowest) mode or all of them and R always returns all of the modes. Both python and R provide a way to handle missing values (NaNs and NAs).

§ 4.5. Standard deviation

§ 4.5.1. Definition

Definition of population and sample standard deviation is provided in [\[P1708R2\]](#) in the equations (2) and (3). However, there is a typo in equation (2).

§ 4.5.2. Users' perspective

To compute standard deviation, a type must support subtraction, multiplication, division by a scalar and square root. This reduces the number of potentially supported types. Both the population standard deviation and the sample standard deviation are commonly used.

§ 4.5.3. Interfaces in other languages

Language	Python	R
Syntax	<pre> statistics.pstdev(data, m=None) statistics.stdev(data, m=None) numpy.std(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>) </pre>	<pre> sd(x, na.rm=TRUE) </pre>
Parameters	data: an iterable of at least two real-valued numbers. m: the mean of data (optional parameter) a: array_like Calculate the standard deviation of these values. axis: None or int or tuple of ints, optional Axis or axes along which the standard deviation is computed. d: datatype, optional Type to use in computing the standard deviation. out: ndarray, optional Alternative output array in which to place the result. ddof: int, optional Means Delta Degrees of Freedom. keepdims: bool, optional If this is set to True, the axes which are reduced are left in the result as dimensions with size one.	x: a numeric vector or factor but not a factor coerced by as.double(x). na.rm: a logical value indicating whether NA values should be stripped before the computation proceeds.
Returns	statistics: pstdev returns the population standard deviation stdev returns the sample standard deviation numpy: returns population or sample standard deviation depending on parameters	the sample standard deviation of the values in x
Restrictions	statistics: least two real-valued numbers	vectors of numerics convertible to double
Error handling	statistics: Raises StatisticsError if data is empty. numpy: If the sub-class' method does not implement keepdims any exceptions will be raised. numpy provides a separate function that handles nan (nanstdev)	The standard deviation of one or zero-length vectors
Details	python_doc_statistics python_doc_stdev	R_doc_sd

Python and Matlab support sample and population standard deviation, while R and SAS calculate sample standard deviation only. Python, R, Matlab support excluding NaN values either as a parameter or as a separate function.

§ 4.6. Variance

Definition of variance is provided in [\[P1708R2\]](#).

§ 4.6.1. Users' perspective

Variance is the squared standard deviation. Due to this relationship they are expected to have very similar interfaces.

§ 4.6.2. Interfaces in other languages

Language	Python	R
Syntax	<pre> statistics.pvariance(data, m=None) statistics.variance(data, m=None) numpy.var(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>) </pre>	<pre> var(x, y = N </pre>
Parameters	data: an iterable of at least two real-valued numbers. m: the mean of data (optional parameter) a: array_like Calculate the standard deviation of these values. axis: None or int or tuple of ints, optional Axis or axes along which the standard deviation is computed. d: datatype, optional Type to use in computing the standard deviation. out: ndarray, optional Alternative output array in which to place the result. ddof: int, optional Means Delta Degrees of Freedom. keepdims: bool, optional If this is set to True, the axes which are reduced are left in the result as dimensions with size one.	x: a numeric vector o coercible to numeric y: NULL (default) or compatible dimension na.rm: a logical value should be stripped be use: ignored
Returns	statistics: pvariance returns the population variance variance returns the sample variance numpy: returns population or sample variance depending on parameters	the sample variance c
Restrictions	statistics: least two real-valued numbers	vectors of numerics c
Error handling	statistics: Raises StatisticsError if data is empty. numpy: If the sub-class' method does not implement keepdims any exceptions will be raised. numpy provides a separate function that handles nan (nanstdev)	The variance of a len
Details	python_doc_statistics python_doc_var	R_doc_var

Matlab, Python and SAS provide similar interfaces for variance and standard deviation. Python, R, Matlab support NaN excluding either as a parameter or as a separate function.

§ 5. Grouped statistics

Grouped statistics are expected to provide a way to calculate a group of statistics in one pass (or as fewer passes as possible) over a container.

§ 5.1. Expectations from the users' perspective

The interface for computing a group of statistics is expected to fulfil common requirements stated in section 3 and the following requirements:

- extendable

- a way to pass parameters to every statistic
- information about required statistics handled in types
- consistent with stand-alone statistical functions

§ 5.2. Users' perspective

Mean, variance, skewness and kurtosis are commonly described as (raw, central or standardized) moments of order 1, 2, 3, and 4. Consequently, it might be reasonable to implement a function for computing moments of a chosen order. There are diverging opinions whether these moments should be available for group computation alongside statistics of different origin, such as quantiles. Proponents of common treatment stress user-friendliness and the statistical notions of location (mean, median) and scale (variance, inter quantile range) parameters. Opponents highlight implementation-based difficulties and point out conceptual differences in the way these statistics are constructed.

§ 6. Some use-cases

1. A user wants to calculate a mean of a vector of doubles.
2. A user wants to calculate a mean of a list of a custom numeric type that support addition as well as division by integer.
3. A user wants to calculate a mean of a vector of floats, where missing data are marked as values below -90.
4. A user wants to calculate a mean of a vector of floats using a double as a return value and internal representation during calculations (for precision).
5. A user wants to calculate mean, variance and standard deviation of a sample in a computationally effective way.
6. A user wants to calculate mean, variance and standard deviation of a sample in a computationally effective way, but her biggest concern is numerical stability of the calculation.
7. A user wants to calculate a median of values stored in a map if keys are in a certain range.
8. A user wants to calculate quantiles 0.25, 0.5 and 0.75 of a container of floats excluding NaNs that are in the data.
9. A user wants to calculate percentils of a sample (i.e. quantiles with order 0.01, 0.02, ..., 0.99).
10. A user wants to calculate percentils of a sorted sample.

§ 7. Summary

In my opinion, the solution proposed in [\[P1708R2\]](#) needs a major revision to fulfill the expectations stated in the previous sections.

§ 8. Document history

- **R0**, 2020-02-13: Initial version.

§ 9. Acknowledgements

I would like to express my gratitude to everyone who provided comments on the use cases and acknowledge discussions in SG6, SG14, SG18 and SG19.

Jolanta Opara's work was supported by Mobica Limited sp. z o.o. Oddział w Polsce.

§ References

§ Normative References

[P1708R0]

Richard Dosselmann, Michael Wong. [Simple Statistics functions](#). 17 June 2019. URL: <https://wg21.link/p1708r0>

[P1708R1]

Michael Wong. [Simple Statistical Functions](#). 10 October 2019. URL: <https://wg21.link/p1708r1>

[P1708R2]

Michael Wong, Micheal Chiu, Richard Dosselmann, Eric Niebler, Phillip Ratzlof, Vincent Reverdy. [Simple Statistical Functions](#). 10 January 2020. URL: <https://wg21.link/p1708r2>

§ Informative References

[HYNDMAN+FAN_1996]

Rob J. Hyndman; Yanan Fan. [Sample Quantiles in Statistical Packages](http://www.amherst.edu/media/view/129116/original/Sample+Quantiles.pdf). The American Statistician, Vol. 50, No. 4 (Nov., 1996), pp. 361-365. URL: <http://www.amherst.edu/media/view/129116/original/Sample+Quantiles.pdf>

[N1668]

Paul A Bristow. [A Proposal to add Mathematical Functions for Statistics to the C++ Standard Library](https://wg21.link/n1668). 11 August 2004. URL: <https://wg21.link/n1668>

[UNUR_2016]

A. Sinan Unur. [Descriptive Stats with C++ and Boost](http://www.nu42.com/2016/12/descriptive-stats-with-cpp-boost.html). URL: <http://www.nu42.com/2016/12/descriptive-stats-with-cpp-boost.html>